

DOCUMENTO

Ordenação e Dois Ponteiros

ordenacao-e-dois-ponteiros.md

Sumário

Ordenação e Dois Ponteiros 4

Parte 1 — Ordenação 4

1. O que é ordenar	4
2. A função sort	4
O que são begin() e end()	5
Ordenando um vetor de strings	5
Ordenando um array comum (estilo C)	6
3. A regra padrão: o operador <	6
Ordem decrescente	6
4. Ordenação customizada: o comparador	6
O contrato do comparador	7
Decrescente com comparador	7
Lambda: o comparador escrito na hora	8
Critérios de desempate (ordenação em vários níveis)	8
5. Cuidados com o comparador	9
Nunca devolva true para elementos iguais	9
Seja consistente entre os critérios	9
O sort não é estável	9
6. Aplicação: o problema 2693 (Van)	10
O enunciado, em resumo	10
A dica	10

Parte 2 — Dois Ponteiros 11

7. O que é a técnica	11
O que ela substitui	11
O preço: os dados precisam ter uma ordem	11
8. As variantes	12
8.1 Ponteiros que se aproximam (das pontas para o meio)	12
8.2 Ponteiros no mesmo sentido (janela deslizante)	12
8.3 Um ponteiro em cada sequência	12

O que as três têm em comum	13
9. A variante das pontas para o meio, em detalhe	13
Por que descartar um elemento não perde a resposta	13
Simulando	14
10. Aplicação: o problema 2422 (Soma das Casas)	14
O enunciado, em resumo	14
A dica	15
11. Erros comuns	15
Resumo rápido	15

Ordenação e Dois Ponteiros

Este material continua os anteriores de Introdução ao C++, `vector` e Busca Binária. Aqui tratamos de duas ferramentas que aparecem o tempo todo em programação competitiva:

1. **Ordenação** com a função `sort`, incluindo a **ordenação customizada**, que é quando você decide o que significa “vir antes”;
2. A técnica dos **dois ponteiros**, que resolve em uma única passada problemas que pareciam exigir testar todas as combinações possíveis.

As duas andam juntas com mais frequência do que parece: quase sempre a técnica de dois ponteiros só funciona porque os dados estão **ordenados**.

Parte 1 — Ordenação

1. O que é ordenar

Ordenar é reorganizar os elementos de uma sequência seguindo uma regra de “quem vem antes de quem”. O caso mais familiar é o crescente:

```
antes:  7  2  9  1  5
depois: 1  2  5  7  9
```

Ordenar é útil por si só (o problema pede a lista em ordem), mas o motivo mais importante é outro: **muitos algoritmos só funcionam, ou só ficam simples, em cima de dados ordenados**. A busca binária é um exemplo que você já viu. Os dois ponteiros, que veremos na Parte 2, são outro.

Você poderia escrever o seu próprio algoritmo de ordenação — *bubble sort*, *selection sort*, *insertion sort*. Eles são ótimos para entender o assunto, mas são lentos demais para vetores grandes e, na prática, ninguém os usa em prova. O C++ já traz uma ordenação pronta, muito rápida e testada: a função `sort`.

2. A função `sort`

Ela vem de `<algorithm>` (ou, como usamos aqui, de `<bits/stdc++.h>`).

```
#include <bits/stdc++.h>
using namespace std;

int main () {
    vector<int> v = {7, 2, 9, 1, 5};

    sort(v.begin(), v.end());

    for (int x : v) cout << x << ' ';    // 1 2 5 7 9
}
```

cpp

A chamada `sort(v.begin(), v.end())` diz: “ordene do começo ao fim do vetor `v`”.

O que são `begin()` e `end()`

`v.begin()` é um **iterador** apontando para o **primeiro** elemento e `v.end()` é um iterador apontando para a posição **logo depois do último**. Ou seja, o intervalo é *fechado no início e aberto no fim*: entra o primeiro, não entra o “um-depois-do-último”.

Isso permite ordenar só um pedaço do vetor:

```
vector<int> v = {7, 2, 9, 1, 5};

sort(v.begin(), v.begin() + 3);    // ordena só os 3 primeiros

// v agora é: 2 7 9 1 5
```

cpp

Ordenando um vetor de strings

Funciona igual, e a ordem usada é a **lexicográfica** (ordem de dicionário, comparando caractere por caractere):

```
vector<string> nomes = {"Kaio", "Emanuel", "Hugo", "Fabricio"};
sort(nomes.begin(), nomes.end());
// Emanuel Fabricio Hugo Kaio
```

cpp

ATENÇÃO

A comparação de `string` usa a tabela ASCII, e nela **todas as letras maiúsculas vêm antes de todas as minúsculas**. Então `"Zebra" < "ana"` é verdadeiro. Se o problema pedir ordem alfabética ignorando maiúsculas, você precisa tratar isso você mesmo.

Ordenando um array comum (estilo C)

```
int a[5] = {7, 2, 9, 1, 5};
sort(a, a + 5);
```

cpp

Aqui `a` faz o papel de “começo” e `a + 5` o de “um depois do fim”.

3. A regra padrão: o operador `<`

Quando você chama `sort` com dois argumentos, ele usa o operador `<` do tipo para decidir a ordem. É por isso que:

- `int`, `double`, `char` → ordem numérica crescente (para `char`, a ordem ASCII);
- `string` → ordem de dicionário;
- `pair<A, B>` → compara primeiro o `first`; **só se der empate**, compara o `second`;
- `tuple` e `vector` → mesma ideia, elemento por elemento, da esquerda para a direita.

O comportamento do `pair` é muito útil e vale memorizar:

```
vector<pair<int, string>> v = {{3, "Ana"}, {1, "Zoe"}, {3, "Bia"}, {1, "Ana"}};
sort(v.begin(), v.end());
// (1, Ana) (1, Zoe) (3, Ana) (3, Bia)
```

cpp

Repare: os pares com `first = 1` vieram primeiro; entre eles, o desempate foi pelo nome.

Ordem decrescente

Duas formas fáceis:

```
// 1) ordena crescente e inverte
sort(v.begin(), v.end());
reverse(v.begin(), v.end());

// 2) usa o comparador pronto greater
sort(v.begin(), v.end(), greater<int>());
```

cpp

Esse terceiro argumento é justamente o assunto da próxima seção.

4. Ordenação customizada: o comparador

E quando o critério não é simplesmente “o menor primeiro”? Por exemplo:

- ordenar alunos pela nota, da maior para a menor;
- ordenar palavras pelo **tamanho**, e não pela ordem alfabética;
- ordenar pontos pela distância até a origem;
- ordenar pessoas por idade e, em caso de empate, pelo nome.

Nesses casos passamos um **terceiro argumento** para o `sort`: uma função de comparação, chamada de **comparador**.

```
sort(inicio, fim, comparador);
```

cpp

O contrato do comparador

O comparador é uma função que recebe **dois elementos**, digamos `x` e `y`, e devolve `true` se, e somente se, `x` deve vir antes de `y` no resultado final.

É só isso. Você não precisa dizer “quanto” `x` é menor, nem devolver `-1`, `0` ou `1` como em outras linguagens. É uma pergunta de sim ou não: `x` vem antes de `y`?

```
bool porTamanho (const string &x, const string &y) {  
    return x.size() < y.size();    // a menor string vem antes  
}  
  
int main () {  
    vector<string> v = {"banana", "uva", "maracuja", "pera"};  
    sort(v.begin(), v.end(), porTamanho);  
    // uva pera banana maracuja  
}
```

cpp

Repare que passamos `porTamanho` **sem os parênteses**: não estamos chamando a função, estamos entregando a função em si para o `sort` chamar quando precisar.

Decrescente com comparador

```
bool decrescente (int x, int y) {  
    return x > y;    // o maior vem antes  
}  
  
sort(v.begin(), v.end(), decrescente);
```

cpp

Olhando os dois exemplos, sai uma regrinha prática que resolve a maioria dos casos:

Você quer...	Escreva no comparador
crescente	<code>return x < y;</code>
decrescente	<code>return x > y;</code>
crescente por um campo	<code>return x.campo < y.campo;</code>
decrescente por um campo	<code>return x.campo > y.campo;</code>

Lambda: o comparador escrito na hora

Criar uma função com nome só para usá-la uma vez é chato. O C++ permite escrever a função **dentro** da chamada do `sort`. Isso se chama **lambda**:

```
sort(v.begin(), v.end(), [](const string &x, const string &y) {
    return x.size() < y.size();
});
```

cpp

Anatomia de uma lambda:

```
[ ]      (const string &x, const string &y)    { return ...; }
^          ^                                   ^
captura   parâmetros (os dois elementos)      corpo da função
(vazia aqui)
```

- `[]` é a **lista de captura**. Vazia significa “essa função não usa nenhuma variável de fora”. Se precisar usar (por exemplo, um outro vetor declarado na `main`), escreva `[&]`, que dá acesso a tudo que está em volta.
- Os parâmetros são os dois elementos a comparar. Usamos `const T &` por dois motivos: `&` evita copiar o objeto a cada comparação (rápido) e `const` garante que não vamos alterá-lo sem querer (seguro).
- O corpo devolve `true` se o primeiro deve vir antes do segundo.

Não há nenhuma diferença de comportamento entre a lambda e a função com nome: é só uma forma mais curta de escrever a mesma coisa, e é o que se usa no dia a dia da competitiva.

Crítérios de desempate (ordenação em vários níveis)

Esse é o padrão mais importante deste material. Quando o problema pede “ordene por A; em caso de empate, por B; em caso de empate ainda, por C”, o comparador fica assim:

```
[](const T &x, const T &y) {  
    if (x.a != y.a) return x.a < y.a;    // critério 1  
    if (x.b != y.b) return x.b < y.b;    // critério 2 (desempate)  
    return x.c < y.c;                    // critério 3 (último desempate)  
}
```

cpp

A lógica linha a linha:

1. Se os campos `a` são **diferentes**, já dá para decidir: devolvemos a comparação de `a` e a função termina ali.
2. Se chegamos na segunda linha, é porque `x.a == y.a` (empate no primeiro critério). Então repetimos o raciocínio com `b`.
3. Na última linha não há mais o que testar, então devolvemos a comparação direta do último critério.

Esse formato “se diferente, decide; se igual, desce para o próximo” é o esqueleto que você vai reescrever em dezenas de problemas.

5. Cuidados com o comparador

O `sort` confia que o seu comparador define uma ordem coerente. Se ele não definir, o resultado pode ser uma ordenação errada — ou até um travamento ou falha de segmentação do programa, porque o algoritmo interno acaba acessando posições fora do vetor. Vale prestar atenção nestes pontos:

Nunca devolva `true` para elementos iguais

```
// ERRADO  
return x.custo <= y.custo;
```

cpp

Se `x` e `y` tiverem o mesmo custo, esse comparador diz “x vem antes de y” e, ao ser chamado na ordem trocada, diz também “y vem antes de x”. Isso é uma contradição, e o `sort` pode se perder. **Sempre use `<` ou `>`, nunca `<=` ou `>=`.**

Seja consistente entre os critérios

Num comparador de vários níveis, todos os testes devem ter a mesma direção lógica de leitura. O erro comum é esquecer um `if` e comparar um campo que já estava sendo usado como desempate.

O `sort` não é estável

“Estável” quer dizer que elementos considerados iguais pelo comparador mantêm entre si a ordem original de entrada. O `sort` **não garante isso**. Se o problema exigir esse comportamento, existem duas saídas:

- usar `stable_sort(inicio, fim, comparador)`, que garante a estabilidade;

- ou acrescentar ao comparador um último desempate que torne a ordem única (por exemplo, pelo índice original ou pelo nome).

A segunda opção é a preferida, porque deixa explícito no código qual é o critério real.

6. Aplicação: o problema 2693 (Van)

O enunciado, em resumo

Uma van vai entregar q alunos em casa. Para cada aluno temos o **nome**, a **região** onde mora (uma letra: L, N, O ou S) e o **custo** até a casa dele. A rota deve entregar os alunos ordenados:

1. pelo **custo**, do menor para o maior;
2. em caso de empate, pela **região**, em ordem alfabética;
3. em caso de empate ainda, pelo **nome**, em ordem alfabética.

A saída é a lista dos nomes, um por linha, nessa ordem. A entrada tem vários casos de teste, até o fim do arquivo (EOF).

```
5
Samuel O 1
Fabricio L 1
Emanuel S 3
Kaio S 20
Hugo N 90
```

Samuel e Fabricio empatam no custo; quem decide é a região. Emanuel, Kaio e Hugo têm custos distintos e saem só pelo custo.

A dica

Leia de novo a lista de critérios do enunciado: são três, um servindo de desempate para o anterior. Isso é exatamente o padrão de vários níveis da seção 4. Pense em como manter nome, região e custo **juntos** enquanto o `sort` embaralha as posições (a mesma seção mostra como), e escreva o comparador traduzindo os três critérios na mesma ordem em que o enunciado os lista.

Dois detalhes que costumam derrubar essa questão:

- `regiao` é um `char`; pergunte-se se o `<` entre letras maiúsculas já dá a ordem alfabética pedida.
- não há um número de casos de teste nem um marcador de fim. Lembre-se do `while (cin >> q)` do material de Introdução ao C++.

Parte 2 — Dois Ponteiros

7. O que é a técnica

Dois ponteiros é uma forma de percorrer uma sequência usando **duas variáveis de índice ao mesmo tempo**, em vez de uma só.

O nome “ponteiro” aqui é só um apelido: são duas variáveis `int` comuns, que guardam posições do vetor. Chamamos de ponteiro porque cada uma “aponta” para um elemento, e a ideia central da técnica é **como essas duas posições caminham**.

A regra que dá nome e poder à técnica é esta:

Cada um dos dois índices caminha **sempre no mesmo sentido e nunca volta atrás**.

Parece uma restrição pequena, mas é ela que faz toda a diferença. Como nenhum dos dois retrocede, cada um percorre o vetor no máximo uma vez de ponta a ponta. O resultado é que o problema inteiro se resolve em **uma única passada pelos dados**, mesmo quando a pergunta parecia exigir testar todas as combinações.

O que ela substitui

O sintoma típico de um problema que pede dois ponteiros é a solução ingênua com **dois laços aninhados**:

```
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
        // testa o par (i, j)
```

cpp

Esse código está **correto**, mas testa todos os pares possíveis. Com `n = 1.000` elementos são cerca de 500 mil pares — tranquilo. Com `n = 100.000`, são mais de **5 bilhões** de pares. Nenhum computador de prova dá conta disso no tempo limite, e o veredito seria *Time Limit Exceeded*.

Com dois ponteiros, o mesmo problema custa no máximo 100.000 passos.

O preço: os dados precisam ter uma ordem

A técnica não é mágica. Ela só funciona quando existe uma **relação de ordem** entre os dados que permita dizer, com certeza, que um elemento **nunca mais será útil** e pode ser descartado para sempre — é isso que autoriza um ponteiro a avançar sem nunca voltar.

Na prática, isso quase sempre significa: **o vetor está ordenado** (ou você o ordena antes, com o `sort` da Parte 1). É por isso que os dois assuntos deste material andam juntos.

8. As variantes

“Dois ponteiros” é um nome guarda-chuva. Existem três formatos principais, e vale reconhecer os três, porque a diferença entre eles está em **para onde cada ponteiro anda**.

8.1 Ponteiros que se aproximam (das pontas para o meio)

Um índice começa no **início** do vetor e o outro no **fim**, e eles caminham um em direção ao outro até se encontrarem.

```

    esq →                                ← dir
      v                                  v
[  1    2    3    5    8   11   14   20   27  ]

```

A cada passo você olha o par formado pelos dois e decide **qual dos dois descartar**, movendo só aquele ponteiro. O laço tem sempre esta cara:

```

int esq = 0, dir = n - 1;
while (esq < dir) {
    // olha o par (v[esq], v[dir]) e decide quem anda
}

```

cpp

Serve para perguntas do tipo: *existe um par com soma k ?*, *qual o par de soma mais próxima de k ?*, *essa palavra é um palíndromo?*, *qual o maior volume de água que cabe entre duas paredes?*

É a variante que detalharemos na Seção 9.

8.2 Ponteiros no mesmo sentido (janela deslizante)

Os dois índices começam no início e **ambos andam para a direita**. O `dir` avança **incluindo** elementos em um trecho; quando esse trecho deixa de satisfazer alguma condição, o `esq` avança **removendo** elementos até voltar a satisfazer. O pedaço entre `esq` e `dir` é chamado de **janela**.

```

      esq      dir
      v        v
[  4  2  1  7  3  5  9  2  ]
    └─ janela ─┘

```

Serve para perguntas sobre **trechos contínuos** (elementos em posições seguidas): *qual o maior trecho cuja soma não passa de x ?*, *qual o menor trecho que contém todas as letras pedidas?*, *qual o maior trecho sem elementos repetidos?*

Repare que aqui também nenhum ponteiro volta: `esq` só cresce, `dir` só cresce.

8.3 Um ponteiro em cada sequência

Quando há **dois vetores ordenados**, colocamos um índice em cada um e sempre avançamos aquele que aponta para o menor valor.

```
A: [ 1  4  7  9 ]      B: [ 2  4  6 ]
    ^                  ^
    i                  j
```

Serve para intercalar duas listas ordenadas em uma só, achar os elementos em comum entre elas, ou descobrir o que existe em uma e não na outra.

O que as três têm em comum

Em todas, a pergunta a se fazer a cada passo é a mesma:

Com a informação que tenho agora, qual ponteiro posso mover com a certeza de não perder a resposta?

Se você consegue responder isso, a técnica se aplica. Se não consegue, é sinal de que os dados ainda não estão na ordem certa — ou de que o problema pede outra coisa.

9. A variante das pontas para o meio, em detalhe

Vamos fechar o zoom na variante mais comum em prova.

O cenário: um vetor **ordenado** e a pergunta *existe um par de elementos cuja soma seja exatamente k?*

A montagem: `esq` na primeira posição (o menor valor) e `dir` na última (o maior valor). Olhamos a soma `v[esq] + v[dir]` e caímos em um de três casos:

Situação	O que fazer	Por quê
soma igual a <code>k</code>	achamos o par, paramos	é a resposta
soma maior que <code>k</code>	<code>dir--</code>	precisamos de uma soma menor
soma menor que <code>k</code>	<code>esq++</code>	precisamos de uma soma maior

Os dois últimos casos merecem atenção: como o vetor está ordenado, **mover `esq` para a direita só aumenta a soma** (pega um valor maior) e **mover `dir` para a esquerda só diminui a soma** (pega um valor menor). Ou seja, o movimento a fazer é sempre o único que empurra a soma na direção certa.

Por que descartar um elemento não perde a resposta

Essa é a parte que vale entender em vez de decorar.

Suponha que `v[esq] + v[dir] > k`, e que por isso decidimos abandonar `v[dir]`. Estamos afirmando que `v[dir]` **não forma o par com ninguém**. Por quê? Porque `v[esq]` é, neste momento, o **menor**

valor ainda disponível. Se nem somado ao menor de todos a soma coube em k — ela já passou —, com qualquer outro parceiro, que seria maior ou igual a $v[\text{esq}]$, a soma seria ainda maior. Logo, $v[\text{dir}]$ é inútil e pode ser jogado fora com segurança.

O raciocínio do outro lado é o espelho: se $v[\text{esq}] + v[\text{dir}] < k$, então $v[\text{esq}]$ somado ao **maior** valor disponível ainda ficou pequeno demais; com qualquer outro parceiro ficaria menor ainda. Então quem é inútil é $v[\text{esq}]$.

IMPORTANTE

Toda essa justificativa depende de o vetor estar **ordenado**. É a ordenação que garante que “mover **esq** para a direita aumenta a soma” e “mover **dir** para a esquerda diminui a soma”. Sem ela, descartar um elemento seria um chute — e a resposta poderia estar justamente no que foi descartado.

Simulando

Vetor 1 2 3 5 8 11 14 20 27, procurando soma $k = 19$:

esq	dir	v[esq]	v[dir]	soma	comparação com 19	ação
0	8	1	27	28	maior	dir--
0	7	1	20	21	maior	dir--
0	6	1	14	15	menor	esq++
1	6	2	14	16	menor	esq++
2	6	3	14	17	menor	esq++
3	6	5	14	19	igual	achou: 5 14

Seis passos, em vez dos 36 pares que os dois laços aninhados testariam.

10. Aplicação: o problema 2422 (Soma das Casas)

O enunciado, em resumo

Uma rua tem N casas, cada uma com um número. Dois brinquedos foram escondidos em duas casas **distintas**, e a dica é que a soma dos números dessas duas casas é k . O enunciado garante que **não existe outro par de casas com essa mesma soma**, ou seja, a resposta é única. Imprima os dois números, o menor primeiro.

```
4          casas: 1 2 3 5, K = 8  →  3 5
1 2 3 5
8
```

A dica

Preste atenção em três coisas que o enunciado diz: em que **ordem** os números das casas são dados, o tamanho de `N`, e a exigência de que as casas sejam **distintas**. A primeira decide se você precisa ou não do `sort`; a segunda decide se a solução de dois laços aninhados passa no tempo; a terceira decide qual é a condição de parada do seu laço (`esq < dir` ou `esq <= dir`?). Com isso resolvido, é a variante da seção 9.

Mais dois cuidados:

- confira a ordem em que as coisas aparecem na entrada: `k` vem **depois** da lista de casas. Ler na hora errada dá *Wrong Answer* sem nenhuma pista.
- o número de cada casa chega a 1.000.000.000. A soma de duas casas cabe em um `int`? Faça a conta e decida se precisa de `long long`.

11. Erros comuns

Em ordenação:

- Usar `<=` ou `>=` no comparador. Sempre `<` ou `>`.
- Ordenar vetores paralelos separadamente, desmontando a correspondência entre eles. Use `struct` ou `pair`.
- Esquecer que `sort(v.begin(), v.end())` ordena até `end()`, que é *um depois do último* — passar `v.end() - 1` deixa o último elemento fora.
- Esperar ordem alfabética “de gente” em `string` com letras maiúsculas e minúsculas misturadas.
- Passar `comparador()` com parênteses quando é uma função sua. Passe só o nome: `comparador`.

Em dois ponteiros:

- Aplicar a técnica em um vetor **não ordenado**. É a causa número um de respostas erradas.
- Usar `esq <= dir` quando o problema exige elementos distintos.
- Mover os dois ponteiros de uma vez, ou mover o ponteiro errado — vale sempre refazer a pergunta “eu quero aumentar ou diminuir a soma?”.
- Fazer um ponteiro voltar atrás. Se o seu código precisa disso, não é mais dois ponteiros, e a garantia de uma única passada se perde.
- Esquecer o `break` (ou o `return`) depois de achar a resposta, e acabar imprimindo mais de um par.

Resumo rápido

Quero...	Uso
Ordem crescente	<code>sort(v.begin(), v.end())</code>
Ordem decrescente	<code>sort(v.begin(), v.end(), greater<int>())</code>
Ordem própria	<code>sort(v.begin(), v.end(), lambda)</code>
Manter a ordem original entre empates	<code>stable_sort(...)</code>
Inverter um vetor	<code>reverse(v.begin(), v.end())</code>

Comparador: devolve `true` se o primeiro argumento deve vir **antes** do segundo. Nunca use `<=`.

Desempate em vários níveis:

```
if (x.a != y.a) return x.a < y.a;
if (x.b != y.b) return x.b < y.b;
return x.c < y.c;
```

cpp

Dois ponteiros: dois índices que caminham pelo vetor e **nunca voltam atrás**, resolvendo em uma passada o que os dois laços aninhados fariam testando tudo. Variantes: das pontas para o meio, janela deslizante, e um ponteiro em cada sequência.

Esqueleto das pontas para o meio (soma alvo em vetor ordenado):

```
int esq = 0, dir = n - 1;
while (esq < dir) {
    int soma = v[esq] + v[dir];
    if (soma == alvo) { /* achou */ break; }
    if (soma > alvo) dir--;
    else esq++;
}
```

cpp

A pergunta que dispara a técnica na prova: “os dados estão (ou podem ser colocados) em ordem, e eu preciso achar um par ou um trecho com alguma propriedade?” Se a resposta for sim, provavelmente dá para trocar os dois laços aninhados por dois ponteiros e uma única passada.